7. Additional Items for Client Clarification

The following items were identified during verification that may benefit from client input. These do not block Phase 1 delivery but are relevant for tournament execution and production deployment.

7.1 Tournament Script Bug Fix (BUG-01) Severity: HIGH — Prevents tournament from working correctly Issue: scripts/run_kairos_tournament.sh passes config/profile as positional arguments (line 133), but main.cpp expects -c and -p flags (getopt). This means the tournament always uses the default config.cfg regardless of the per-mode override, so all 7 rounds run with the same scheduler mode. Fix: Change line 133 from: ./Kairos_server "$tmp_config" "$PROFILE" & to: ./Kairos_server -c "$tmp_config" -p "$PROFILE" & Question for client: May we apply this fix and re-run verification? This is a trivial one-line change that aligns the script with the documented server interface.

Regarding BUG-01: Tournament Script Fix, follow these authoritative instructions:

1. Authorization to Fix: You are authorized to apply the fix immediately. Change line 133 in scripts/run_kairos_tournament.sh to use the -c and -p flags as specified.
2. Crucial Correction: This bug likely explains why your previous results showed identical acceptance rates for FCFS, EDF, and DQN. If the server was not receiving the specific configuration file for each mode, it was defaulting to a single scheduler (likely FCFS) for every round.
3. Re-run Verification: After applying this fix, you must re-run the full tournament.
4. Tournament Parameters: To ensure the results are publishable, the re-run must follow the "Stress Tournament" parameters:
    - Volume: 1,000 requests.
    - Intensity: High arrival rate ($\lambda = 5.0$).
    - Asymmetry: 50/50 mix of strong and weak clients ($\theta_c \geq 3.0$).
    - Filters: Set VFT_SAFETY_MARGIN = 1.0 to allow the schedulers to manage the congestion.

Summary:
Apply the fix. This is the "missing link" that will finally allow the system to demonstrate the performance delta between DQN/SA and the baselines. We expect to see a massive improvement in the Ψ (Psi) metric once the correct scheduler modes are actually being loaded.

7.2 Poisson Harness Response Parsing (OBSERVATION-01)
The poisson_test_harness.py (line 315) parses rejection responses looking for "REJECTED_VFT:SUGGESTED_SLO:" but the server sends "REJECTED_404:SUGGESTED_SLO:" (KairosServer.cpp:393). This mismatch means the suggested SLO value is never extracted by the harness, though this is a cosmetic issue — rejections are still counted correctly.
Question for client: Should the harness response parsing be updated to match the server's rejection format, or should the server format be changed to match the harness?

Regarding OBSERVATION-01: Poisson Harness Response Parsing, follow these technical instructions:

1. Decision: Update the Harness to match the Server
The poisson_test_harness.py must be updated to parse the 400-series error codes. We will not change the server format, as the use of REJECTED_404 is the technically correct implementation of the Sequential Exit strategy (C-07) and matches our established error-code standard for system congestion.

2. Technical Requirement
Update line 315 in scripts/poisson_test_harness.py to recognize the differentiated rejection codes:

- Tier 3 (VFT/Congestion): Parse REJECTED_404:SUGGESTED_SLO:<value>.
- Tier 1/2 (Practicality/Asymmetry): Ensure the harness also handles REJECTED_403 (though no SLO is suggested for impractical requests).

3. Scientific Reasoning
While the mismatch is currently "cosmetic" for counting, the successful extraction of the SUGGESTED_SLO is required for the Evaluation section of the paper. We need this data to analyze the "Wait-Time Delta"—the difference between what the client wanted and what the system could actually provide. This is a critical metric for proving the accuracy of our Virtual Finish Time (VFT) estimations and our negotiation logic.

Summary for Task List:

- Modify poisson_test_harness.py: Change the string-match pattern from REJECTED_VFT to REJECTED_404.
- Verify Extraction: Run a short high-load test ($\lambda = 5.0$) and ensure the Python summary logs show the correct suggested SLO values.
- Documentation: Ensure the README.md lists these error codes (403, 404) so the user understands the rejection types.

Technical Grade: 10/10. This ensures that our "Negotiation" data is correctly captured by the benchmarking tools. Thank you for catching this discrepancy.

7.3 Pre-processing Log Arrival_TS (OBSERVATION-02) In the CSV log output from the test run, pre-processing jobs (type='p') have Arrival_TS=0 because they are generated internally by the replenisher, not from client requests. This is functionally correct but may affect post-hoc analysis (analyze_psi.py) since Psi depends on Arrival_TS for TAT calculation. Question for client: Should pre-processing jobs be excluded from the Psi metric calculation, or should Arrival_TS be set to the start timestamp for internal jobs?

Regarding OBSERVATION-02: Pre-processing Log Arrival_TS, follow these technical instructions to maintain the mathematical integrity of the research metrics:

1. Decision: Exclude Pre-processing Jobs from

Ψ Calculation: The Ψ (Psi) metric must only be calculated using Online Inference jobs (J_{on}).

2. Technical Reasoning

- Definition of Ψ: As defined in our formal problem statement, Ψ measures the Quality of Service (QoS) from the client's perspective.
- Avoid Double-Counting: The "cost" of preprocessing is already implicitly captured in the Turnaround Time (TAT) of the inference job. If the replenisher is slow or a request has to wait for a file to be generated, the TAT of the J_{on} job increases, which correctly lowers the Ψ value. Including J_{pre} as a separate entry in the denominator would mathematically double-count the latency for a single successful result.
- Internal Nature: Proactive J_{pre} jobs are internal background tasks. They do not have a client-facing "Arrival Time."

3. Implementation Requirements

A. Logging Logic (Logger.cpp):

- Action: For internal pre-processing jobs (type='p'), set Arrival_TS equal to the Start_TS.
- Purpose: This ensures the CSV remains valid and prevents "zero" or "null" values that could crash post-processing scripts. For these rows,
- TAT will simply equal the execution time (e_{pre}).

B. Analysis Logic (analyze_psi.py):

- Action: Update the script to filter out all jobs where Job_Type == 'p' before calculating the final Ψ.
- Calculation: $\Psi = \frac{\sum \chi (\text{only for } J_{on} \text{ rows})}{\sum TAT (\text{only for } J_{on} \text{ rows})}$

Summary for Task List:

- Modify Logger.cpp: Set Arrival_TS = Start_TS for preprocessing jobs to ensure data sanity.
- Modify analyze_psi.py: Add a filter to calculate Ψ based only on online inference request rows (J_{on}).
- Verify: Ensure that the total number of inputs in your Ψ report matches the total number of requests sent by the Poisson harness.

As far as I know, his preserves the scientific accuracy of our results. Ψ is a metric for the Service Success, not for the Internal Replenishment. Your suggestions are also welcome here.